

Technical University of Applied Sciences Würzburg-Schweinfurt (THWS)
Faculty of Computer Science and Business Information Systems

Master Thesis

Great Research About My Favourite Topic

submitted for the completion of degree

**Master of Science
in
Artificial Intelligence**

Your Name

Submitted on: December 8, 2025

Supervisor: Prof. Dr. Magda Gregorova
Second Examiner: Prof. Dr. C D

Abstract (en)

A single paragraph summarizing the most important points of your thesis. This can be more than one paragraph but should not be too long. You usually write this as the very last thing of your thesis.

Abstract (de)

We are in germany so include a translation.

Acknowledgment

Here you can thank your friends who helped you throughout the studies, your parents for supporting you generally, if you wish your supervisor or any other profs and generally everybody you value.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	1
1.3	Research Goals	2
1.4	Research Questions	2
2	Systematic Literature Review	4
2.1	Phase 1 — Planning	4
2.1.1	Review Protocol	4
2.2	Phase 2 — Conducting the Review	11
2.2.1	Execution of the Search Strategy	11
2.2.2	Removing of Duplications	11
2.2.3	Study Selection	11
3	Text to Vector Generation with Neural Path Representation	13
4	NeuralSVG	18
	Bibliography	18
	List of Figures	22
	Appendix	23
	Great table.	23
	Declaration on oath	24
	Consent to plagiarism check	25

1 Introduction

1.1 Motivation

Scalable Vector Graphics (SVG) play an essential role in modern digital design due to their scalability, compactness, and support for semantic editing. Unlike raster images, SVGs describe visual content through structured, text-based geometry using paths, shapes, and hierarchical groupings, allowing for precise control, reusability, and easy modification.

In recent years, machine learning and deep learning models have been applied to the generation of SVGs from various input modalities, such as text prompts, raster images, and sketches. Open-source research has produced a variety of approaches, including sequence models, transformer-based architectures, and differentiable optimization methods, each proposing different strategies for representing and producing vector graphics.

Given this diversity, it becomes relevant to study how these open-source models differ in their architectures, internal representations, and the structure of the SVGs they generate. Understanding these aspects can provide a clearer picture of how SVGs are represented and produced computationally, and what factors may influence the quality and editability of the resulting graphics.

1.2 Problem Statement

While multiple open-source models exist for SVG generation, they employ heterogeneous approaches in terms of architecture, data representation, and output structure. Some models treat SVGs as sequences of drawing commands, others as hierarchical objects, and still others as continuous geometric parameters optimized through differentiable rendering.

This diversity makes it challenging to compare how different approaches handle SVG generation and how these choices influence the final output. Furthermore, the evaluation

of SVG quality often depends on various factors, such as path simplicity, structural organization, and editability, which are not always measured in the same way.

Therefore, the problem addressed in this research is to systematically describe, compare, and analyze open-source SVG generation methods, focusing on their architectures, internal representations, and output characteristics, in order to understand how these design choices relate to the quality and structure of the generated SVGs.

1.3 Research Goals

1. To explain and analyze the core properties, structures, and components of SVG graphics as a foundation for understanding their representation and generation.
2. To collect, review, and categorize existing open-source methods for SVG generation across various input modalities.
3. To investigate the types of deep learning and machine learning architectures employed in SVG generation models and analyze their design principles.
4. To examine how these architectures represent SVG data internally and how their outputs are transformed into standard SVG files.
5. To explore and evaluate existing approaches, metrics, and benchmarks for assessing the quality of generated SVGs.
6. To assess and compare the output quality of current SVG generation models based on the identified evaluation criteria.

1.4 Research Questions

1. What are the main properties and structures that define an editable and high-quality SVG?
2. What open-source methods currently exist for generating SVGs from different input modalities?
3. What types of deep learning and machine learning architectures are models used for SVG generation?

4. What kinds of structure do these models produce?
5. How can the quality and editability of generated SVG be evaluated?
6. What is the observed quality of SVG outputs across different models?
7. What makes open source model suitable for testing?
8. Which open-source SVG generation models are most suitable for testing and comparison?
9. How do the selected open-source SVG generation models perform in terms of defined criteria (eg. editability, quality and practical usage)?

2 Systematic Literature Review

Kitchenham’s guidelines were originally developed for evidence-based software engineering and have become one of the most widely adopted standards for conducting systematic literature reviews in computer science and technical research disciplines [1]. The methodology provides detailed, domain-specific procedures for planning, searching, selecting, and synthesizing studies in areas where research outputs are often heterogeneous—such as algorithmic models, machine learning architectures, or engineering pipelines. Unlike biomedical-focused protocols such as PRISMA [2], which were designed primarily for clinical trials and human-subject research with standardized quantitative outcomes, Kitchenham addresses challenges unique to computational research: diversity of study designs, inconsistent terminology, non-uniform evaluation metrics, and the frequent presence of code-based or algorithmic artifacts instead of comparable statistical datasets. Similarly, frameworks like SALSA [3] emphasize high-level structuring but do not prescribe the step-by-step operational procedures needed for reproducibility in technical fields.

For this thesis—situated within deep learning, machine learning, and vector-graphics generation—a Kitchenham-style SLR is needed because the literature consists of diverse model architectures, varied input modalities, and different evaluation criteria. Kitchenham’s protocol supports this complexity by providing structured phases (planning, conducting, and reporting) that guide the reviewer through formulating rigorous research questions, defining search strategies, conducting selection criteria, assessing quality, data extraction and performing narrative synthesis. This structure minimizes bias, increases reproducibility, and ensures that the review captures the full range of relevant deep-learning and vectorization approaches.

2.1 Phase 1 — Planning

2.1.1 Review Protocol

Databases

- arXiv
- Google Scholar
- ACM Digital Library

Search Strings/Keywords

SVG Generation

"SVG generation"
"scalable vector graphics" AND generation
"SVG generative models"
"vector graphics generation"
"SVG generation" AND deep learning
"SVG generation" AND machine learning

Raster-to-Vector Vectorization

"raster-to-vector" AND deep learning
"image-to-vector" AND neural network
"image vectorization" AND machine learning
"deep vectorization"
"differentiable vector graphics"

Text-to-SVG

"text-to-SVG"
"text-to-vector graphics"

Structure of SVG, Editability and Quality

"SVG Basics"
"SVG fundamentals"
"Scalable Vector Graphics"

"SVG editability"
"editable SVG"
"SVG path simplification"
"layered SVG" AND generation"

Survey and Review Searches

"SVG" AND "systematic literature review"
"vector graphics" AND survey
"vector image" AND review
"image vectorization" AND survey
"graphic generation" AND review
"SVG Generation" AND benchmark
"image vectorization" AND benchmark

Study Selection Criteria

Inclusion Criteria:

- **Language and Publication:** Written in English and published in a peer-reviewed journal, conference, or workshop.
- **Time Range:** Published between 2020–2025 (post-emergence of modern SVG generative models such as DeepSVG).
- **Domain Relevance:** Falls within computer vision, computer graphics, AI, machine learning, or deep learning.
- **Reproducibility:** Provides an open-source implementation or references a publicly available repository. Availability of pre-trained weights is preferred, but source code alone is sufficient if the method is reproducible.
- **SVG Output Requirement:** Describes a model, method, or architecture that directly generates SVG or structured vector outputs, including:
 - Bézier curves,
 - SVG `<path>` commands,
 - layered or hierarchical SVG structure,

- vector strokes or SVG-like stroke sequences,
- structured outputs suitable for editability analysis.
- **Input Modalities:** Supports at least one of:
 - image-to-SVG vectorization,
 - text-to-SVG generation,
 - multimodal SVG generation,
- **Architectural Transparency:** Provides sufficient technical detail to extract the model architecture and internal SVG representation (e.g., decoder design, transformer layers, vector tokenization, differentiable rasterization, or geometric parameterization).
- **Evaluation:** Provides quantitative metrics and qualitative SVG examples that allow analysis of structural quality and editability.

Exclusion Criteria:

- **No Vector Output:** The method does not generate vector or SVG outputs (pixel-only image generation).
- **Non-ML Methods:** Purely rule-based or classical algorithmic vectorization methods without machine learning or deep learning.
- **Closed-Source Systems:** No publicly available code, weights or sufficient technical detail for reproduction.
- **Insufficient Evaluation:** Papers lacking SVG evaluation, qualitative vector examples or structural analysis of outputs.
- **Out-of-Scope Vector Work:** Studies focused on vector editing, rendering, compression, markup optimization, or stylistic modification without SVG generation.
- **Domain Mismatch:**
 - handwriting synthesis,
 - font generation,

- datasets,
- sketch generation not intended for SVG graphics.
- **Temporal Exclusion:** Studies published before 2020, except when cited for background context only.
- **LLM Restriction:** Exclude proprietary LLMs that generate SVG text but do not provide architecture, training details, or reproducibility.

Study Selection Procedure

- Initial records
- Duplicate removal
- Title/abstract screening
- Full-text assessment
- Final included studies

Quality Assessment Criteria

Following the recommendations by Kitchenham for evidence-based software engineering [1, 4], each primary study must be assessed for quality, the assessment criteria must be explicitly described, applied consistently, and the overall quality of each study should be reported (e.g., high, medium, or low quality). The four Kitchenham categories used in this review are reporting quality, methodological rigor, reproducibility, and result validity. The criteria are listed below.

The following Kitchenham-style quality criteria were applied to all studies that passed the full-text screening. Each criterion was evaluated as **Yes = 1** or **No = 0**. The total score was then used to classify studies as high, medium, or low quality.

Reporting Quality

- The study states its research aim clearly.
- The model or method architecture is described in enough detail.

- The SVG/vector representation (paths, strokes, Béziers, etc.) is explained.

Methodological Rigor

- The method is appropriate for SVG/vector generation.
- The evaluation procedure is described clearly.

Reproducibility

- The study provides open-source code or enough detail to reproduce results.

Result Validity

- Quantitative metrics or qualitative SVG examples are provided.

Scoring and Classification Each criterion was scored as either **Yes = 1** or **No = 0**. The total score determined the study's quality classification:

- **6–7 points:** High quality
- **4–5 points:** Medium quality
- **0–3 points:** Low quality (may be excluded)

Kitchenham recommends that “studies should be classified as high, medium or low quality based on quality criteria” [1].

Data Extraction Plan According to Kitchenham's guidelines for evidence-based software engineering, the purpose of data extraction is to systematically collect all information required to answer the research questions in a consistent and reproducible way. Kitchenham emphasises that a predefined data extraction form must be created *during the planning phase* to ensure uniformity and reduce subjective variation among reviewers [1].

Predefined Data Extraction Form Kitchenham states that researchers should develop a structured data extraction form before conducting the review. This prevents ad-hoc decisions and ensures that all necessary information is collected consistently across all studies [1].

Alignment with Research Questions Kitchenham notes that extracted data must be driven directly by the review’s research questions. The form should therefore include only fields that support answering the research questions or performing the planned synthesis [1].

Information to Extract The guidelines specify that extraction should capture both bibliographic information and structured study attributes, including study context, aims, methodology, data used, experimental setup, and results. Kitchenham emphasises that the level of detail must be sufficient for later comparison and synthesis [1].

Reviewer Consistency Kitchenham recommends that extracted data should be validated—ideally by a second reviewer—to minimise bias and ensure consistency. If this is not possible, reviewers should re-check their own extraction for consistency [1].

Structured (Not Free-Form) Fields Kitchenham advises the use of structured or semi-structured fields to minimise subjectivity and ensure that comparable information is extracted across studies. Examples include fixed fields for model architecture, datasets, evaluation metrics, and SVG output structure [1].

Avoiding Unnecessary Information Only data needed for answering the research questions or for synthesis should be extracted. Kitchenham warns that recording irrelevant details increases effort and contributes no value to the final review [4].

Ensuring Comparability Across Studies Finally, Kitchenham highlights that the extraction process must ensure that the information gathered from different studies is comparable and suitable for aggregation or narrative synthesis [4]. Using a consistent extraction form enables systematic comparison of SVG generation models.

2.2 Phase 2 — Conducting the Review

2.2.1 Execution of the Search Strategy

The search strings defined in Phase 1 were executed across the selected databases (Google Scholar, arXiv, and ACM Digital Library) to identify relevant literature on SVG generation and vector-based deep learning methods. Searches were performed using the conceptual string groups defined in the search strategy, with database-specific syntax adjustments where necessary. All records retrieved by these searches were exported and documented before any filtering or removal of duplicates. Table 2.1 summarises the number of studies returned by each search group and database.

Table 2.1: Search Log Summary Across Databases and Conceptual Groups

Search Group	Google Scholar	arXiv	ACM DL	Total
SVG Generation	14	12	4	30
Raster-to-Vector	7	10	2	19
Text-to-SVG	7	6	5	18
Surveys/Benchmarks/Reviews	6	3	1	10
SVG Fundamentals / Background	3	3	2	8
Total Initial findings	37	34	14	85

2.2.2 Removing of Duplications

After merging all search results, duplicate records—including repeated titles and preprint–conference duplicates—were removed. This resulted in 72 unique studies.

2.2.3 Study Selection

- Stage 1: Title, Keywords and Abstract screening.

After removing duplicates, **72** unique studies remained. A brief title and abstract screening excluded works clearly unrelated to SVG generation (e.g., sketch-only models, raster methods, classical vectorization, or closed-source LLMs), resulting in **37** papers proceeding to full-text review.

- Stage 2: Full-text screening

A full-text assessment was conducted on the **37** studies that passed the title and abstract screening. Each paper was examined for reproducibility, architectural transparency, and whether it genuinely produced SVG or structured vector outputs as defined by the inclusion criteria. In total, **10** studies were excluded at this stage: four lacked any open-source implementation, two were benchmark papers evaluating proprietary LLMs such as GPT, Gemini, or Claude, two used non-reproducible LLM-based pipelines without sufficient methodological detail, and one relied on a purely rule-based vectorization approach. Following this eligibility check, **27** studies remained for quality assessment.

Table 2.2: Reasons for Exclusion During Full-Text Screening

Exclusion Reason	Count
No open-source implementation	4
Proprietary LLM benchmark studies (GPT, Gemini, Claude)	2
Non-reproducible LLM-based pipeline	2
Purely rule-based vectorization method	2
Total Excluded	10

Table 2.3: Study Selection Summary Across Screening Stages

Selection Stage	Number of Studies
Initial search results (all databases)	85
After duplicate removal	72
After title and abstract screening	37
After full-text screening	27

3 Text to Vector Generation with Neural Path Representation

Table 3.1: Metadata

Field	Value
Title	Text-to-Vector Generation with Neural Path Representation
Authors	Peiying Zhang, Nanxuan Zhao, Jing Liao
Year	2024
Venue	SIGGRAPH 2024 (accepted)
DOI/URL	0.48550/arXiv.2405.10317

Table 3.2: Problem & Task

Case	Notes
Task	"Text-to-Vector (T2V) generation, where the model learns to generate individual vector paths instead of the entire SVG at once."
Input modality	"A single text prompt (T)."
Output	"SVG consisting of a set of paths ($\text{SVG} = \{\text{Path}_1, \text{Path}_2, \dots, \text{Path}_m\}$), each represented by a latent code, color, and transformation parameters."
Domain	"Vector graphics, digital art, clipart, animation, and graphic design applications."
Motivation	"Complex vector graphics are composed of simple paths. Previous T2V methods often produce intersecting or jagged paths due to lacking geometry constraints. This paper proposes a method to overcome these limitations by learning paths individually with a neural path representation."
Goal	"The goal is to generate an SVG that aligns with the semantics of the text prompt while exhibiting desirable path properties and layer-wise structures consistent with human perception. "

Table 3.3: Method / Model Overview

Field	Value
Architecture	Two-stage pipeline: Stage A: Neural Path Representation Learning (Dual-branch VAE). Stage B: Text-driven Neural Path Optimization(Optimization with Variational Score Distillation and Layer-wise Vectorization).
Stage A	VAE trained on FIGR-8-SVG dataset (200,000 vector icons). Step 1: Start with vector paths from the dataset (control points for each path). Step 2: Rasterize each path into an image to capture its visual appearance. Step 3: Encode vectors (sequence branch) and raster images (image branch) into latent space. Step 4: Share a common latent vector z between both branches. Step 5: Decode back to reconstruct both the original vector paths and the rendered images. Each path is encoded independently; no fixed number of paths.
Stage B	Text-driven optimization aligns latent vectors $z_1, \dots, z_m$ to text prompt via Variational Score Distillation (VSD), followed by layer-wise optimization and path refinement.
Vector Representation	Each path represented by $\theta_i = (z_i, C_i, Tr_i)$ including latent code, color, transformation. Latent dimension $d_F = 24$.
Loss Function	VAE training: $L = \lambda_{cfr}L_{cfr} + \lambda_{img}L_{img} + \lambda_{kl}L_{kl}$. Stage 1: VSD loss. Stage 2: Layer-wise loss $L_{lyr} = L_{CLIP} + \lambda_{IoU}L_{IoU}$.
Training Details	VAE training: Adam optimizer, LR=0.001, linear warmup/decay, dropout 0.1, trained for 100 epochs.
Optimization Techniques	VSD(diffusion model as guidance, updates these z vectors so that, when decoded into SVG paths, the resulting image matches the text prompt), layer-wise vectorization(ensures that paths don't overlap badly and follow a clear hierarchical structure), path simplification(removes unnecessary paths, low-opacity paths, and merges overlapping paths), CLIP/diffusion guidance(provides text-image similarity score). SDS used in baselines (VectorFusion) for comparison.
Extensions	Supports line art (fine-tuned on sketches), exemplar-based SVG customization, and animation via text-guided optimization.

Table 3.4: Experimental Setup and Evaluation

Field	Details
Metrics Used	Smoothness, Simplicity (number of paths), Layer-wise Semantics, FID(visual quality), CLIP score(text-image correspondence) and Human preference.
Baselines Compared	T2I + LoRA + Potrace, T2I + LIVE, CLIPDraw, VectorFusion.
Evaluation Types	Quantitative metrics (numerical scores) and qualitative metrics (user study / human preference).
General Performance	Proposed T2V-NPR model improves path smoothness, semantic alignment, text-image correspondence, and human-perceived SVG quality over baselines.

Table 3.5: Ablation Study

Variant	Description	Smooth $\uparrow$	Simp $\downarrow$	$S_{img}\uparrow$
w/o NPR	Neural Path Representation is disabled; paths are directly optimized via control points without learned latent geometry. This leads to unstable, jagged, and intersecting curves.	0.5271	40	0.9941
Sequence Only (VAE)	Only the sequence (geometry) encoder-decoder of the dual-branch VAE is used; the raster/image modality is removed, reducing appearance-awareness of paths.	0.8116	40	0.9910
w/o VSD (Stage 1)	Stage-1 text alignment is performed without Variational Score Distillation (VSD), meaning no stable latent-space alignment to the text prompt. Produces overly complex SVGs with many redundant paths.	0.7896	380	—
SDS (Stage 1)	Score Distillation Sampling (SDS) replaces VSD for text-driven optimization. SDS gives reasonable paths but tends to oversmooth and reduces diversity.	0.8070	40	0.9932
VSD Only (Stage 2)	Only VSD is used in Stage-2 optimization, without layer-wise structural refinement. Produces smooth paths but less organized layer structure.	0.8194	40	—
w/o Layer-wise (Stage 2)	Stage-2 optimization removes the layer-wise structural loss; paths are optimized globally only. Reduces clarity and topological ordering between layers.	0.7924	40	0.9929
Ours (Full Model)	Full pipeline with NPR, dual-branch VAE, Stage-1 VSD text alignment, and Stage-2 layer-wise refinement. Balanced performance across all metrics.	0.8012	40	0.9926

Table 3.6: Limitations

Relies on the generative ability of the diffusion model. Overly detailed text prompts may lead to missing semantic elements (e.g., “ping pong table”, “ashtray”).
Tends to oversimplify shapes with intricate boundaries. Complex edges exceed latent-space capacity, causing smoothing and loss of fine details (e.g., bear’s body contours).
Optimization is iterative and slow. Full pipeline takes 13 minutes for 128 paths on an NVIDIA 3090, whereas T2I vectorization baselines operate in seconds.
In NeuralSVG paper, mentioned that T2V with NPR approach are unavailable.

4 NeuralSVG

Table 4.1: Metadata

Field	Value
Title	NeuralSVG: Converting Text to Vector Graphics via Joint Shape and Color Optimization
Authors	Bowen Zhang, Zixuan Lu, Li-Yi Wei, Fang-Lue Zhang
Year	2025
Venue	ACM Transaction on Graphics
DOI/URL	10.48550/ARXIV.2501.03992

Table 4.2: Method / Model Overview

Field	Value
Vector Representation	SVG is modeled as N Bézier curves; each curve has 12 control-point coordinates + 1 color vector. You can visualize this as a 2D diamond composed of 4 Bézier segments (12 total points). Each shape also has a scalar index used for positional encoding.
Positional Encoding	Shape index i is mapped to a high-dimensional embedding using Random Fourier Features (64 frequencies $\rightarrow$ 128D vector, sin/cos pairs). This allows the system to encode periodic geometric patterns across shapes.
Network Architecture	Two MLPs are used: • $\mathbf{MLP}_{\text{pos}}(v_i) \rightarrow \mathbb{R}^{12 \times 2}$ outputs the Bézier control points. • $\mathbf{MLP}_{\text{c}}(v_i) \rightarrow [0, 1]^3$ outputs RGB color (sigmoid). The input v_i is the 128D Fourier feature embedding.
Initialization Pipeline	1. Text prompt. 2. Stable Diffusion generates a raster image. 3. Attention map detects important regions. 4. Sample coarse shapes. 5. Convert them into convex polygons / Bézier curves. 6. Extract initial colors from raster pixels $\rightarrow (p_t^{\text{init}}, c_t^{\text{init}})$ for each shape. 7. Train MLPs with L2 losses so they start from a reasonable SVG guess. This avoids chaotic optimization at the start.
Initialization Loss	$L_{\text{pos}}(i) = \ \mathbf{MLP}_{\text{pos}}(v_i) - p_t^{\text{init}}\ _2^2$ $L_{\text{c}}(i) = \ \mathbf{MLP}_{\text{c}}(v_i) - c_t^{\text{init}}\ _2^2$
Training / Optimization	1. MLPs reconstruct initial SVG. 2. SVG is rasterized differentiably. 3. Score Distillation Sampling (SDS) compares rasterized SVG with diffusion-model output. 4. Diffusion model predicts noise. 5. Noise gives gradient signal. 6. Backpropagation updates $\mathbf{MLP}_{\text{pos}}$ and $\mathbf{MLP}_{\text{c}}$.
SDS Text Alignment	SDS uses Stable Diffusion to push the SVG toward the semantics of the text prompt. Unlike NPR, there is no latent VAE; gradients directly update the SVG primitives.
Ordered Representation	A nested dropout technique randomly drops shapes during training, ensuring that: • important shapes become stable and survive, • unimportant shapes fade away, • scene is simplified and layers become meaningful.
Differentiable Rendering	The neural renderer (MLPs) outputs colors at sampled pixel locations, enabling continuous rasterization and end-to-end optimization.
Extra Controls	Supports color palette constraints and aspect-ratio constraints.

Table 4.3: Experimental Setup & Evaluation

Field	Details
Metrics Used	• CLIP cosine similarity (SVG $\downarrow$ - $\downarrow$ text alignment) • R-Precision (SVG matches correct prompt among distractors) • Qualitative comparisons
Baselines Compared	VectorFusion, SVGDreamer, Text-to-Vector and NiVEL.
Evaluation Types	Quantitative metrics (CLIP, R-Precision) and qualitative results.
General Performance	NeuralSVG outperforms baselines in text alignment (CLIP), improves recognizability, and produces cleaner vector shapes and achieve accurate visual with less shapes.

Table 4.4: Ablation Study

Ablation study in NeuralSVG is limited: the only quantitative comparison provided is between model trained with and without the dropout technique. Even in this case, the difference in cumulative CLIP similarity scores is relatively small (e.g., 0.26 vs. 0.27), suggesting that the impact of dropout on semantic alignment, while visually noticeable, is minor according to this metric. But qualitatively diff. is recognizable. Other design choices, such as using separate MLPs for geometry and color or directly optimizing control points, are only discussed qualitatively.

Table 4.5: Limitations
No limitations mentioned.

Bibliography

- [1] Barbara Kitchenham. *Procedures for Performing Systematic Reviews*. en. Technical Report. Keele University and NICTA, 2004.
- [2] David Moher, Alessandro Liberati, Jennifer Tetzlaff, Douglas G. Altman. “Preferred reporting items for systematic reviews and meta-analyses: the PRISMA statement”. en. In: *BMJ* 339 (July 2009). Publisher: British Medical Journal Publishing Group Section: Research Methods & Reporting, b2535.
- [3] *A typology of reviews: an analysis of 14 review types and associated methodologies - Grant - 2009 - Health Information & Libraries Journal - Wiley Online Library*.
- [4] Barbara Kitchenham, O. Pearl Brereton, David Budgen, Mark Turner, John Bailey, Stephen Linkman. “Systematic literature reviews in software engineering – A systematic literature review”. en. In: *Information and Software Technology* 51.1 (Jan. 2009), pp. 7–15.

List of Figures

Appendix

Great table.

Headline 1	Headline 2
Titel 1	Description 1
Titel 2	Description 2

Table 1: Overview in a great table.

Declaration on oath

I hereby certify that I have written my master thesis independently and have not yet submitted it for examination purposes elsewhere. All sources and aids used are listed, literal and meaningful quotations have been marked as such.

December 8, 2025, Your Name

Consent to plagiarism check

I hereby agree that my submitted work may be sent to PlagAware (<https://my.plagaware.com/>) in digital form for the purpose of checking for plagiarism and that it may be temporarily (max. 5 years) stored in the database maintained by PlagScan as well as personal data which are part of this work may be stored there.

Consent is voluntary. Without this consent, the plagiarism check cannot be prevented by removing all personal data and protecting the copyright requirements. Consent to the storage and use of personal data may be revoked at any time by notifying the faculty.

December 8, 2025, Your Name